Module 4

1. Implementation of NER (NameEntity Recognition)

2. Explore various parsing techniques

3. Explore various Word Sense Disambiguation Techniques

1. Implementation of Named Entity Recognition (NER)

NER identifies and classifies named entities in text such as person, organization, location, date, etc.

Resource Used
• spaCy
• NLTK

Step-by-Step Process

Step 1: Use Preprocessed Text

Use text that has already undergone earlier NLP steps such as:
• Tokenization
• Stop word removal
• POS tagging

Example sentence:

Sample Text: SUBJECTIVE:, This 23-year-old white female presents with complaint of allergies. She used to have allergies when she lived in Seattle but she thinks they are worse here. In the past, she has tried Claritin, and Zyrtec. Both worked for short time but then seemed to lose effectiveness. She has us...

Step 2: Identify Candidate Words

Analyze the sentence to detect words likely to represent entities.

Examples:

- 23-year-old
- Seattle
- Zyrtec

These words typically represent:
• Person names
• Locations
• Dates

Step 3: Apply NER Model

Apply the Named Entity Recognition model to the text.

The model analyzes:
• Context of the word
• Part-of-speech tags
• Capitalization patterns

Code:

```
if 'df' in locals() and not df.empty:
    sample_text = df['transcription'].iloc[0]
    sample_text_short = sample_text[:300] + "..."

    doc_ner = nlp(sample_text_short)

    print(f"Sample Text: {sample_text_short}\n")
    print(f"{'Entity':<30} | {'Label':<10} | {'Explanation'}")
    print("-" * 85)
    for ent in doc_ner.ents:
        explanation = spacy.explain(ent.label_) if spacy.explain(ent.label_)
else "Medical Entity (DRUG/DISEASE)"
        print(f"{ent.text[:28]:<30} | {ent.label_:<10} | {explanation}")
else:
    print("Dataframe not available.")
```

Step 4: Classify Entities

Each detected entity is assigned a specific label.

Common labels:

Label	Meaning
PERSON	Name of a person
ORG	Organization
GPE	Location
DATE	Date

Step 5: Extract Named Entities

Extract all detected entities along with their labels.

Output:

```
========================================
1. Implementation of Named Entity Recognition (NER)
========================================
Sample Text: SUBJECTIVE:,  This 23-year-old white female presents with complaint of allergies.  She
used to have allergies when she lived in Seattle but she thinks they are worse here.  In the past, she
has tried Claritin, and Zyrtec.  Both worked for short time but then seemed to lose effectiveness.
She has us...

Entity                          | Label      | Description
-------------------------------------------------------------------------------------
23-year-old                     | DATE       | Absolute or relative dates or periods
Seattle                         | GPE        | Countries, cities, states
Claritin                        | PERSON     | People, including fictional
Zyrtec                          | LANGUAGE   | Any named language
```

NER helps:
• Information extraction
• Question answering systems
• Search engines
• Chatbots

2. Explore Various Parsing Techniques

Parsing determines the grammatical structure of a sentence.

Two main techniques are commonly explored:

1. Constituency Parsing
2. Dependency Parsing

Resource Used
• NLTK
• spaCy

Step-by-Step Process

Step 1: Use POS Tagged Sentence

Take a sentence that has Part-of-Speech tags.

Example:

Sentence: SUBJECTIVE:, This 23-year-old white female presents with complaint of allergies.

Step 2: Define Grammar Rules

Grammar rules define how words combine to form phrases.

Example rules:

S → NP VP
NP → DT NN
VP → V NP

Meaning:

• Sentence → Noun Phrase + Verb Phrase
• Noun Phrase → Determiner + Noun
• Verb Phrase → Verb + Noun Phrase

Step 3: Apply Parsing Algorithm

The parser analyzes the sentence structure using grammar rules.

It builds a parse tree showing hierarchical structure.

Code:

```
sentences = list(doc_ner.sents)
if sentences:
    sentence_dep = sentences[0].text
else:
    sentence_dep = "The patient was discharged today."

doc_dep = nlp(sentence_dep)

print(f"Sentence: {sentence_dep}\n")
print(f"{'Word':<20} | {'Relation (Dep)':<15} | {'Head Word':<20}")
print("-" * 65)
for token in doc_dep:
    print(f"{token.text:<20} | {token.dep_:<15} | {token.head.text:<20}")

# Using basic NLTK regex parser on the same sentence
tokens = nltk.word_tokenize(sentence_dep)
pos_tags = nltk.pos_tag(tokens)

grammar = r"""
  NP: {<DT|PRP\$>?<JJ>*<NN|NNS>} # Noun Phrase
  VP: {<VBD|VBZ|VBP><NP>}      # Verb Phrase
"""
cp = nltk.RegexpParser(grammar)
result = cp.parse(pos_tags)
print("Result Parse Tree:")
print(result)
```

Output:

```
    |     | [ S ] (Sentence)
    ___________|_____________________________________________________________

    |      |    |        |                  |        |        |        |     |        |
  [ NP ]   [:]  [,]    [ NP ]              [ NP ]   [IN]    [ NP ]    [IN]  [ NP ]     [.]

    |      |    |   _____|_______            |        |        |        |     |        |
SUBJECTIVE  :    ,  |    |    | |         presents   with  complaint    of  allergies   .
    /NN            This 23-y white female        /NNS             /NN             /NNS
    |     |     |     /DT /JJ  /JJ   /NN
```

Step 4: Dependency Parsing

Instead of phrase structure, dependency parsing shows relationships between words.

Example sentence:

Sentence: SUBJECTIVE:, This 23-year-old white female presents with complaint of allergies.

Dependencies identified:

Word	Relation (Dep)	Head Word
SUBJECTIVE	ROOT	SUBJECTIVE
:	punct	SUBJECTIVE
,	punct	SUBJECTIVE
	nmod:tmod	SUBJECTIVE
This	det	female
23-year-old	advmod	female
white	amod	female
female	amod	presents
presents	nmod	SUBJECTIVE
with	case	complaint
complaint	nmod	presents
of	case	allergies
allergies	nmod	complaint
.	punct	SUBJECTIVE

Parsing helps:
• Understand sentence structure
• Improve machine translation
• Support grammar checking

3. Explore Various Word Sense Disambiguation Techniques

Word Sense Disambiguation (WSD) identifies the correct meaning of a word based on context.

Example:

"The patient is ready for discharge from the hospital.",

"There was a purulent discharge from the wound."

Possible meanings:
• remove the charge
• pus coming out of the wound

Resource Used
• WordNet
• NLTK

Step-by-Step Process

Step 1: Select an Ambiguous Word

Choose a word that has multiple meanings.

Example:

Discharge

Step 2: Analyze Context

Look at the sentence where the word appears.

Example sentences:

1. "The patient is ready for **discharge** from the hospital.",
2. "There was a purulent **discharge** from the wound."

Step 3: Retrieve Word Senses

Retrieve all possible meanings of the word using lexical resources.

Example senses of discharge:

1. remove the charge from...
2. the pouring forth of a fluid...

Step 4: Apply WSD Technique

Common techniques include:

1. Knowledge-Based Methods

Example: Lesk Algorithm

Matches context words with dictionary definitions.

2. Supervised Learning

Uses labeled training data to learn word meanings.

3. Unsupervised Learning

Clusters contexts to identify possible senses.

Code:

By Lesk Algorithum:

```
ambiguous_word = "discharge"
statements = [
    "The patient is ready for discharge from the hospital.",
    "There was a purulent discharge from the wound."
]

print(f"{'Sentence':<55} | {'Sense':<30} | {'Definition'}")
print("-" * 150)

for sentence in statements:
    tokens = nltk.word_tokenize(sentence)
    sense = lesk(tokens, ambiguous_word)
    if sense:
        print(f"{sentence[:53]:<55} | {sense.name():<30} |
{sense.definition()[:100]}...")
    else:
        print(f"{sentence[:53]:<55} | {'No sense found':<30} | ")
```

By Walker Algorithm:

```
def walkers_wsd(sentence, target):
    tokens = nltk.word_tokenize(sentence.lower())
    pos = nltk.pos_tag(tokens)
    target_tag = next((t[1][0].lower() for t in pos if t[0] == target), 'n')
    wn_pos = {'n': wn.NOUN, 'v': wn.VERB, 'j': wn.ADJ, 'r':
wn.ADV}.get(target_tag, wn.NOUN)

    target_senses = wn.synsets(target, pos=wn_pos)
    if not target_senses: target_senses = wn.synsets(target)

    context_synsets = [s for t in tokens if t != target and len(t) > 2 for s
in wn.synsets(t, pos=wn.NOUN)]

    if not target_senses or not context_synsets:
        return target_senses[0] if target_senses else None

    return max(target_senses, key=lambda s: sum(s.path_similarity(c) or 0 for
c in context_synsets))
```

Step 5: Select Correct Meaning

Based on context, the algorithm selects the most suitable sense.

Example:

Sentence	Correct Meaning
The patient is ready for discharge from the hospital.	Discharge from hospital
The fisherman sat on the river bank	Pus from the wound

By Lesk Algorithm:

Output:

```
========================================
3.1 Explore Lesk Word Sense Disambiguation
========================================
Sentence                                          | Sense                      | Definition
-------------------------------------------------------------------------------------------------------------------------------
The patient is ready for discharge from the hospital.  | discharge.v.04             | remove the charge from...
There was a purulent discharge from the wound.         | discharge.n.06             | the pouring forth of a fluid...
```

By Walker Algorithm:

Output:

```
========================================
3.2 Walker's Word Sense Disambiguation (Similarity Based)
========================================
Sentence                                          | Sense                      | Definition
-------------------------------------------------------------------------------------------------------------------------------
The patient is ready for discharge from the hospital.  | discharge.n.03             | a substance that is emitted or released...
There was a purulent discharge from the wound.         | discharge.n.01             | the sudden giving off of energy...
```

Result:

Hence, Name Entity recognition, various parsing techniques, Word Sense Disambiguation Techniques has been implemented successfully.